2.1 Vision-Based Line Follower

Abstract

This project implements an autonomous navigation pipeline for a Parrot Minidrone using a downward-facing camera feed (120×160 resolution). The system utilizes a MATLAB-based perception algorithm to extract path orientation and execute real-time steering corrections. By employing a localized Region of Interest (ROI) and color-space thresholding, the drone achieves stable trajectory tracking. Furthermore, a terminal logic system is integrated to monitor red pixel density, allowing the drone to autonomously transition to a landing state upon completion of the track or detection of a terminal marker.

I. INTRODUCTION

The objective of this task is to design a robust perception-to-action loop. The drone must interpret raw visual data to maintain its position over a red track. Our approach focuses on converting 2D pixel coordinates into a spatial error signal (e). We utilize a Proportional Control strategy to map this error to a steering angle. Throughout the development, we prioritized computational efficiency to ensure the algorithm remains compatible with the drone's high-frequency flight control requirements.

II. PROBLEM STATEMENT

The system must process a $120 \times 160 \times 3$ RGB image to output:

- **Line Angle:** A numerical steering value representing the deviation from the track.
- **Stop Flag:** A binary trigger (0 or 1) to initiate the landing sequence.

Constraints: 1. The drone must handle curves without losing the track.

2. The system must detect the end of the line (track termination) or a landing circle to stop safely.

III. RELATED WORK

While Hough Transforms are frequently used for line detection in static images, they often struggle with curved paths and high computational latency. Alternatively, PID Control based on infrared sensors is common in ground robotics but lacks the spatial resolution provided by a camera. Our implementation uses **Geometric Centroid Analysis**, which combines the high resolution of vision sensors with the speed of simple algebraic calculations.

IV. INITIAL ATTEMPTS

- **Global Binarization:** Initially, we tried thresholding the entire 120×160 frame. However, shadows and reflections at the top of the frame (near the horizon) created "ghost" pixels that caused the drone to steer erratically.

- **Edge-Based Following:** Attempting to follow only the left or right edge of the red line proved unstable during sharp turns, as the edge would often leave the camera's field of view before the center of the line did.

V. FINAL APPROACH

The final implementation follows a specialized four-stage perception pipeline:

1. ROI Slicing (Spatial Filtering)

We crop the input image to focus only on the bottom 40 rows (Rows 80 to 120).

- **Logic:** By looking only at the path immediately beneath the drone, we ignore distant visual noise and ensure the steering command is based on the drone's current alignment.

2. Relative Color Segmentation

To isolate the red track, we apply a relative difference mask:

$$Red\_Mask = (R > 110) \cap (R > G + 25) \cap (R > B + 25)$$

- **Reasoning:** This formula is superior to absolute thresholding because it ignores "White" or "Grey" pixels (where $R \approx G \approx B$) and specifically targets high-saturation red.

3. Centroid and Error Calculation

We calculate the horizontal mean ($x_{centroid}$) of all pixels identified in the Red_Mask.

- **Error Equation:**

$$Error = x_{centroid} - 80$$

(Where 80 is the center of the 160px width).

- **Steering Law:**

$$Line\_Angle = K_p \times Error$$

(With tuned gain $K_p = -0.007$).

4. Terminal "Presence" Logic

The system performs a continuous "Red Pixel Count" (N).

- **End-of-Track:** If $N < 20$, the drone assumes the track has ended.
- **Landing Circle:** If $N > 1200$, the drone assumes it is directly over the large landing marker.
- **Result:** Both conditions set stop_flag = 1, triggering the descent.

VI. RESULTS AND OBSERVATION

- **Precision:** The 120×160 resolution allowed for an error sensitivity of 0.625% per pixel, providing highly granular feedback to the flight controller.

- **Performance:** By processing only the ROI, the execution time was reduced by approximately 66% compared to full-frame processing.
- **Stability:** The centroid method naturally filters out "salt-and-pepper" noise, as the average position of the line is unaffected by a few stray pixels.

VII. FUTURE WORK

- **Adaptive K_p:** Implementing a variable gain that increases when the centroid moves rapidly (indicating a sharp curve).
- **Temporal Smoothing:** Using a moving average for the line_angle to prevent high-frequency "jitter" in the drone's motors.

CONCLUSION

The task was successfully addressed by designing a "human-approximate" perception system. By utilizing geometric centroids and conditional stop logic, the drone is capable of both precision navigation and autonomous decision-making. This approach demonstrates a balanced priority for computational speed and flight reliability, which is essential for the mission-critical environments explored by Aerial Robotics Kharagpur.